

Algoritmos y Estructuras de Datos.

- Clase 4 - Estructuras de datos lineales -

Tipos de Datos Abstractos (TAD)

- Tipos de Datos
- Listas Enlazadas
- Pilas
- Colas

Punteros

En otros lenguajes, como C o C++, existe un tipo de variable especial llamado puntero, que se comporta como una referencia a una variable, **como es el caso de las variables mutables**.

En Python *no hay punteros* (como los de C o C++), pero todas las variables son referencias a objetos, es decir, la *dirección* de la porción de memoria, en donde el objeto está almacenado. De este modo cuando se asigna una variable a otra, lo que se está asignando es la dirección de la porción de memoria guarda el objeto. Si esa porción de memoria cambia, el cambio se puede ver en todas las variables que apuntan a esa porción.

El uso de '_' in Python

El uso del carácter '_' (guión bajo) tiene significado especial en Python. Podemos utilizar uno o dos guiones bajos para identificar nombres de variables y métodos especiales (aquellos que queremos proteger). Además cada combinación puede de varias connotaciones:

- Simple guión bajo: `_`
El identificador especial `_` se utiliza en el intérprete interactivo para almacenar el resultado de la última evaluación. Cuando no está en modo interactivo, `_` no tiene un significado especial y no está definido. También podemos usarlo para denotar una variable que no usaremos, indicando que es un valor "temporal".
- Simple guión bajo frente al *nombre de variable o método*: `_nombre`
Denota una variable o método para uso interno, estos son accesibles como cualquier otro identificador, excepto cuando importamos un módulo. Es decir, que no serán importados por `from nombre-módulo import *`
- Simple guión bajo detrás al *nombre de variable o método*: `nombre_`
Sin distinción en particular.
- Doble guión bajo frente al *nombre de variable o método*: `__nombre`
Identificadores de clase privados. Los nombres de esta categoría, cuando se utilizan en el contexto de una definición de clase, se reescriben para utilizar una forma desfigurada que ayude a evitar conflictos de nombres entre los atributos *privados* de las clases base y derivadas. Es decir, concatenará el nombre de la clase al identificador.
- Doble guión bajo frente y detrás al *nombre de variable o método*: `__nombre__`
Identificadores definidos por el sistema, conocidos informalmente como nombres **dunder** (double-under). Estos nombres son definidos por el intérprete y su aplicación (incluida la biblioteca estándar). A

continuación podremos ver el método `__init__` al comienzo de una clase. Estos métodos también son conocidos como *métodos mágicos*. Cualquier uso de estos nombres, en cualquier contexto, que no siga un uso explícitamente documentado, está sujeto a que se rompa sin previo aviso.

Pilas

El comportamiento de una *Pila* se puede describir mediante la frase “Lo último que se apiló es lo primero que se usa”. Este método se llama **LIFO** (Last In First Out).

Formalmente un *Pila* es un TAD que tiene las siguientes operaciones:

- `__init__` : Inicializa una pila vacía.
- `push` (apilar): Agrega un nuevo elemento a la Pila
- `pop` (desapilar): Remueve el *tope* de la Pila y lo devuelve. Este es el último elemento que se agregó.
- `is_empty` (está_vacía): Retorna **True** o **False** según si la pila está vacía o no.

Opciones:

- `top` : Retorna el *tope* de la Pila (sin removerlo).

In []:

```
p=Pila()

def top(P):
    if is_empry : pass
    x = P.pop
    P.push(x)
    return x
```

Implementación de Pilas utilizando Listas

Definiremos una clase Pila con un atributo, `items`, de tipo lista, que contendrá los elementos de la pila. El *tope* de la pila se encontrará en la última posición de la lista, y cada vez que se apile un nuevo elemento, se lo agregará al final.

In [9]:

```
class Pila:
    """ Representa una pila con operaciones de apilar, desapilar y
        verificar si está vacía. """

    def __init__(self):
        """ Crea una pila vacía. """
# La pila vacía se representa con una lista vacía
        self.items=[]

    def push(self, x):
        """ Agrega el elemento x a la pila. """
# Apilar es agregar al final de la lista.
        self.items.append(x)

# Desapilar usará el método pop de lista que hace exactamente lo requerido
    def pop(self):
        """ Devuelve el elemento tope y lo elimina de la pila.
            Si la pila está vacía levanta una excepción. """
        try:
            return self.items.pop()
        except IndexError:
            raise ValueError("La pila está vacía")

# El método para indicar si se trata de una pila vacía.
    def is_empty(self):
        """ Devuelve True si la lista está vacía, False si no. """
        return self.items == []
```

In [11]:

```
p = Pila()
p.is_empty()
p.push(1)
p.is_empty()
p.push(5)
p.push("+")
p.pop()
```

```
-----
-----
TypeError                                 Traceback (most recent call
last)
<ipython-input-11-da8b62acb4d9> in <module>
      6 p.push("+")
      7 p.pop()
----> 8 for x in p:
      9     p.pop()

TypeError: 'Pila' object is not iterable
```

Ejercicio: Implementar el método **top**, modificando el método **pop**.

In []:

```
#Ejercicio: Implementar el método **top**, modificando el método **pop**.
```

```
# Retorna el *tope* de la Pila
```

```
## Modificar ##
```

```
def pop(self):  
    """ Devuelve el elemento tope y lo elimina de la pila.  
    Si la pila está vacía levanta una excepción. """  
    try:  
        return self.items.pop()  
    except IndexError:  
        raise ValueError("La pila está vacía")
```

```
def top(P):  
    if is_empry : pass  
    x = P.pop  
    P.push(x)  
    return x
```

In []:

Colas

Todos sabemos lo que es una *Cola*. Este método se llama **FIFO** (First In First Out).

Formalmente un *Cola* es un TAD que tiene las siguientes operaciones:

- `__init__` : Inicializa una cola vacía.
- `enqueue` (encolar o `push`): Agrega un nuevo elemento al final de la Cola
- `dequeue` (desencolar o `pop`): Remueve el primer elemento de la Cola y lo devuelve.
- `is_empty` (está_vacía): Retorna **True** o **False** según si la Cola está vacía o no.

In [12]:

```
class Cola:
    """ Representa a una cola, con operaciones de encolar y
    desencolar. El primero en ser encolado es también el primero en ser desencolado.

    def __init__(self):
        """ Crea una cola vacía. """
# La cola vacía se representa por una lista vacía
        self.items=[]

#El método encolar se implementará agregando el nuevo elemento al final de la lista:
    def encolar(self, x):
        """ Agrega el elemento x como último de la cola. """
        self.items.append(x)

    def desencolar(self):
        """ Elimina el primer elemento de la cola y devuelve su
        valor. Si la cola está vacía, levanta ValueError. """
        try:
            return self.items.pop(0)
        except:
            raise ValueError("La cola está vacía")

#Por último, el método es_vacia, que indicará si la cola está o no vacía.
    def es_vacia(self):
        """ Devuelve True si la cola esta vacía, False si no."""
        return self.items == []
```

In [17]:

```
q = Cola()
q.es_vacia()
q.encolar(1)
q.encolar(2)
q.encolar(5)
q.es_vacia()
q.desencolar()
q.desencolar()
q.encolar(8)
q.desencolar()
q.desencolar()
q.es_vacia()
#q.desencolar()
```

Out[17]:

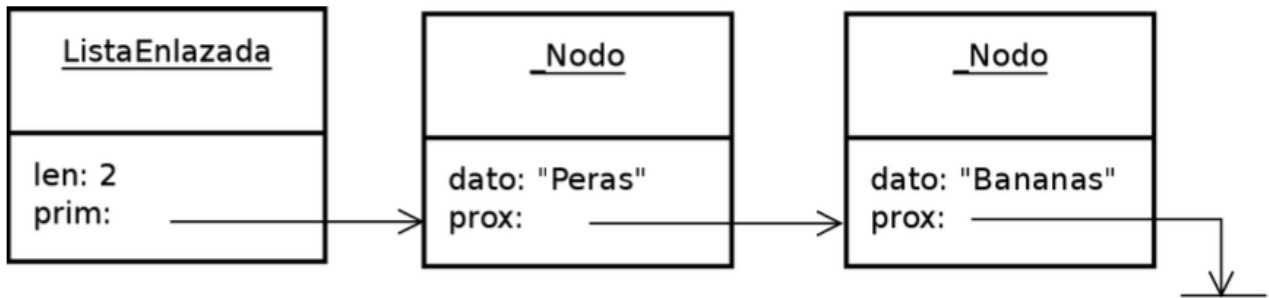
True

Listas, Pilas y Colas Enlazadas

Listas enlazadas

Definimos a continuación las operaciones que inicialmente deberá cumplir la clase ListaEnlazada.

- `__str__`, para mostrar la lista.
- `__len__`, para calcular la longitud de la lista.
- `append(x)`, para agregar un elemento al final de la lista.
- `insert(i, x)`, para agregar el elemento `x` en la posición `i` (levanta una excepción si la posición `i` es inválida).
- `remove(x)`, para eliminar la primera aparición de `x` en la lista (levanta una excepción si `x` no está).
- `pop([i])`, para borrar el elemento que está en la posición `i` y devolver su valor. Si no se especifica el valor de `i`,
- `pop()` elimina y devuelve el elemento que está en el último lugar de la lista (levanta una excepción si se hace referencia a una posición no válida de la lista).
- `index(x)`, devuelve la posición de la primera aparición de `x` en la lista (levanta una excepción si `x` no está).



Definición de la clase `Nodo`:

In [5]:

```
class Nodo(object):
    def __init__(self, dato=None, prox = None):
        self.dato = dato
        self.prox = prox
    def __str__(self):
        return str(self.dato)
```

In []:

```
class ListaEnlazada(object):
    "Modela una lista enlazada, compuesta de Nodos. "
    def __init__(self):
        """Crea una lista enlazada vacía. """
        # prim: apuntará al primer nodo - None con la lista vacía
        self.prim = None
        # len: longitud de la lista - 0 con la lista vacía
        self.len = 0
```

Definición de los métodos mas importantes:

In [32]:

```
ada(object):
    lista enlazada, compuesta de Nodos. "
(self):
    una lista enlazada vacía. ""
    tará al primer nodo - None con la lista vacía
    m = None
    tud de la lista - 0 con la lista vacía
    = 0

(self):
    ve el iterador de la lista. "
    IteradorListaEnlazada(self.prim)

, i = None):
    ina el nodo de la posición i, y devuelve el dato contenido.
    está fuera de rango, se levanta la excepción IndexError. Si no se recibe la posición
    ecibió i, se devuelve el último.
    None:
    self.len - 1
    ión de los límites
    ot (0 <= i < self.len):
    raise IndexError("Índice fuera de rango")
    r, si es el primero,
    r la cabecera de la lista

    = self.prim.dato
    .prim = self.prim.prox
    s los demás elementos, busca la posición

t = self.prim
t = n_ant.prox
in xrange(1, i):
t = n_act
t = n_ant.prox
a el dato y elimina el nodo a borrar
    = n_act.dato
t.prox = n_act.prox
estar 1 de len
.len -= 1
r el valor borrado
ato
```

In [19]:

```
def remove(self, x):
    """ Borra la primera aparición del valor x en la lista.
        Si x no está en la lista, levanta ValueError """
    if self.len == 0:
        # Si la lista está vacía, no hay nada que borrar.
        raise ValueError("Lista vacía")
    # Caso particular, x esta en el primer nodo
    elif self.prim.dato == x:
        # Se descarta la cabecera de la lista
        self.prim = self.prim.prox
    # En cualquier otro caso, hay que buscar a x
    else:
        # Obtiene el nodo anterior al que contiene a x (n_ant) n_ant = self.prim
        n_act = n_ant.prox
        while n_act != None and n_act.dato != x:
            n_ant = n_act
            n_act = n_act.prox
        # Si no se encontró a x en la lista, levanta la excepción
    if n_act == None:
        raise ValueError("El valor no está en la lista.")
# Si encontró a x, debe pasar de n_ant -> n_x -> n_x.prox # a n_ant -> n_x.prox
    else:
        n_ant.prox = n_act.prox
    # Si no levantó excepción, hay que restar 1 del largo
    self.len -= 1
```

In [21]:

```
def insert(self, i, x):
    """ Inserta el elemento x en la posición i.
        Si la posición es inválida, levanta IndexError """
    if (i > self.len) or (i < 0): # error
        raise IndexError("Posición inválida") # Crea nuevo nodo, con x como dato:
    nuevo = _Nodo(x)
    # Insertar al principio (caso particular)
    if i==0:
        # el siguiente del nuevo pasa a ser el que era primero
        nuevo.prox = self.prim
        # el nuevo pasa a ser el primero de la lista
        self.prim = nuevo
        # Insertar en cualquier lugar > 0
    else:
        # Recorre la lista hasta llegar a la posición deseada n_ant = self.prim
        for pos in xrange(1,i):
            n_ant = n_ant.prox
        # Intercala nuevo y obtiene n_ant -> nuevo -> n_ant.prox
        nuevo.prox = n_ant.prox
        n_ant.prox = nuevo
        # En cualquier caso, incrementar en 1 la longitud
    self.len += 1
```

Iteradores

Todas las secuencias pueden ser recorridas mediante una misma estructura (for variable in secuencia), ya que todas implementan el método especial `__iter__`. Este método debe devolver un **iterador** capaz de recorrer la secuencia como corresponda.

En particular, en Python, los iteradores tienen que implementar un método **next** que debe devolver los elementos, de a uno por vez, comenzando por el primero. Y al llegar al final de la estructura, debe levantar una excepción de tipo **StopIteration**.

Métodos requeridos:

- **__iter__**: Crea un objeto "Iterador".
- **next**: Siguiente elemento en la secuencia.

Excepciones:

- **StopIteration**: Nos avisa cuando se a llegado al final de la secuencia.

Es decir que las siguientes estructuras son equivalentes:

In []:

```
for elemento in secuencia:
    # hacer algo con elemento
```

In []:

```
iterador = iter(secuencia)
while True:
    try:
        elemento = iterador.next()
    except StopIteration:
        break
    # hacer algo con elemento
```

Queremos implementar un iterador para la *Lista Enlazada*, la mejor solución implica crear una nueva clase, *_IteradorListaEnlazada*, que implemente el método *next()* de la forma apropiada.

In [6]:

```
class _IteradorListaEnlazada(object):
    " Iterador para la clase ListaEnlazada "

    def __init__(self, prim):
        """ Constructor del iterador.
            prim es el primer elemento de la lista. """
        self.actual = prim

    def __next__(self):
        """ Devuelve uno a uno los elementos de la lista. """
        if self.actual == None:
            raise StopIteration("No hay más elementos en la lista")
        # Guarda el dato
        dato = self.actual.dato
        # Avanza en la lista
        self.actual = self.actual.prox
        # Devuelve el dato
        return dato
```

In [51]:

```
def __iter__(self):
    " Devuelve el iterador de la lista. "
    return _IteradorListaEnlazada(self.prim)
```

In [7]:

```
l = ListaEnlazada()
l.insert(0,1)
l.insert(0,2)
l.insert(0,3)
for valor in l:
    print(valor)
```

3
2
1

Ejercicio: Escribir los métodos `__iter__`, `__str__` y `__len__` para la lista.

In [44]:

Clase Pasada

Módulo numpy

Su Documentación completa está disponible aqui: <http://www.numpy.org> (<http://www.numpy.org>).

Particularmente interesante para problemas mathematicos. Nos ofrece un nuevo tipo de dato "arreglo" (array), tambien llamados "vectores", cuando todos los elementos son del mismo tipo.

In []:

```
import numpy as np
```

In []:

```
v=np.array([[1,1],[1,1]])# Crea un arreglo de 2 x 2 (es decir, una matriz).
                        # v= [ 1 | 1
                        #      1 | 1 ]

print(v)

print([[1,1],[1,1]])

type(v)
```

```
In [ ]:
```

```
v+=v  
print(v)
```

```
In [ ]:
```

```
v=np.array([[1,2],[3,4]])  
print(v)  
print(v[0][1]) # Podemos indexar los elementos de un array  
v[0][1]=5  
print(v)  
w=v # Los arrays son objetos "mutables".  
w[0][1]=3  
print(w, '\n', v)
```

```
In [ ]:
```

```
np.array([[True,2.0],[0+1j,0.1],[3+7*1j,-np.pi]]) # La *función* array normalizara l  
# que ingresamos por parametros.
```

Además tenemos provisto el tipo *Matriz*.

```
In [ ]:
```

```
M=np.matrix([[0,1j],[1j,0]]) # Tipo : matrix  
print(M.conjugate()) # Retorna w/ conjugado de la matriz.  
print(M*M)
```

Sobrecarga de operadores/comandos (Overloading)

La "sobrecarga" de una función, operador, método, ya definido nos permite cambiar su significado. Por ejemplo, nos gustaria re-definir el operador "suma" "+", para nuestra nueva clase de objetos.